# Esquema Relacional em SQL

Conta (id_conta, username, email, senha, data_nascimento, pais, saldo)

```
create table conta(
    id_conta int primary key,
    username varchar(20) not null,
    email varchar(40) not null unique,
    senha varchar(20) not null,
    data_nascimento date not null,
    pais varchar(40) not null,
    saldo decimal(10,2) not null default 0
            check (saldo >= 0)
);
```

Amizade (id_solicitante, id_destinatario, status)

```
create table amizade(
    id_solicitante int not null,
    id_destinatario int not null,
    status  varchar(10) not null default 'pendente',
            check (status in ('pendente', 'aceita', 'recusada')),
    check (id_solicitante != id_destinatario),
    primary key (id_solicitante, id_destinatario),
    foreign key (id_solicitante) references conta(id_conta) on delete cascade,
    foreign key (id_destinatario) references conta(id_conta) on delete cascade
);
```

Desenvolvedora (id_desenvolvedora, nomed, pais)

```
create table desenvolvedora(
    id_desenvolvedora int not null primary key,
    nomed varchar(50) not null unique,
    pais varchar(50) not null
);
```

Genero (id_genero, nomeg)

```
create table genero(
    id_genero int primary key,
    nomeg varchar(30) not null unique
);
```

Jogo (id_jogo, id_desenvolvedora, titulo, preco)

```
create table jogo(
    id_jogo int primary key,
    id_desenvolvedora int not null,
    titulo varchar(50) not null unique,
    preco decimal(10,2) not null
            check (preco >= 0),
    foreign key (id_desenvolvedora) references desenvolvedora(id_desenvolvedora)
);
```

Jogo_Genero (id_jogo, id_genero)

```
create table jogo_genero(
    id_jogo int not null,
    id_genero int not null,
    primary key (id_jogo, id_genero),
    foreign key (id_jogo) references jogo(id_jogo) on delete cascade,
    foreign key (id_genero) references genero(id_genero) on delete cascade
);
```

Pedido (id_pedido, id_conta, data_pedido)

```
create table pedido(
    id_pedido int primary key,
    id_conta int not null,
    data_pedido date not null,
    foreign key (id_conta) references conta(id_conta) on delete cascade
);
```

Item_Pedido (id_pedido, id_jogo, preco_pago)

```
create table item_pedido(
    id_pedido int not null,
    id_jogo int not null,
    preco_pago decimal(10,2) not null
            check (preco_pago >= 0),
    primary key (id_pedido, id_jogo),
    foreign key (id_pedido) references pedido(id_pedido) on delete cascade,
    foreign key (id_jogo) references jogo(id_jogo)
);
```

Biblioteca(id_conta, id_jogo, horas_jogadas, ultima_sessao)

```
create table avaliacao(
    id_jogo int not null,
    id_conta int not null,
    nota    int not null
            check (nota in (1, 2, 3, 4, 5)),
    comentario varchar(255),
    data_avaliacao date not null,
    primary key (id_jogo, id_conta),
    foreign key (id_conta, id_jogo) references biblioteca(id_conta, id_jogo) on delete cascade
);
```

Avaliacao (id_jogo, id_conta, nota, comentario, data_avaliacao)

```
create table avaliacao(
    id_jogo int not null,
    id_conta int not null,
    nota    int not null
            check (nota in (1, 2, 3, 4, 5)),
    comentario varchar(255),
    data_avaliacao date not null,
    primary key (id_jogo, id_conta),
    foreign key (id_jogo) references jogo(id_jogo) on delete cascade,
```

```
    foreign key (id_conta) references conta(id_conta) on delete cascade
);
```

# Dicionário de Dados

- Conta (id_conta, username, email, senha, data_nascimento, pais, saldo)

→ Propósito: representar uma conta de um usuário do sistema
→ Atributos: id_conta serve como o identificador único do elemento, username, e-mail, senha, data_nascimento são todos dados cadastrais, com e-mail sendo ***unique*** e uma chave alternativa, e saldo é o saldo para uso na loja.
→ Chaves:
Primária: id_conta
Alternativa: e-mail
→ Outras restrições:

```
saldo decimal(10,2) not null default 0
        check (saldo >= 0)
```

Garante que o saldo não será negativo, e é instanciado em 0, por padrão.

- Amizade (id_solicitante, id_destinatario, status)

→ Propósito: representa um pedido de amizade entre 2 usuários.
→ Atributos: id_solicitante e id_destinatario são as contas que solicita e que recebe o convite, respectivamente. Status é a condição de aceite do pedido.
→ Chaves:
Primária: (id_solicitantre, id_destinatário) – uma relação de amizade entre dois usuários é única.
Estrangeiras: id_solicitante e id_destinatario ambas referenciam a tabela *conta*.
→ Outras restrições:

```
status  varchar(10) not null default 'pendente',
        check (status in ('pendente', 'aceita', 'recusada')),
```

Garante que uma amizade tem somente 3 estados, e que por padrão é instanciada como ‘pendente’.

```
check (id_solicitante != id_destinatario),
```

Invalida amizades do tipo (u1, u1).

`on delete cascade` garantem que a amizade será desfeita se qualquer uma das contas for apagada.

- Desenvolvedora (id_desenvolvedora, nomed, pais)

→ Propósito: representar as empresas que desenvolvem os jogos do catálogo para permitir filtro na loja,
→ Atributos: tem um identificador único, um nome, que também é único, e um país de origem.
→ Chaves:
Primária: id_desenvolvedora
Alternativa: nome

- Genero (id_genero, nomeg)

→ Propósito: representa um gênero que pode ser atribuído a um jogo. É modelado numa tabela separada de jogo porque um jogo pode ter mais de um gênero. Nome também é ***unique*** para evitar

duplicatas.
→ Atributos: um identificador único e o nome do gênero.
→ Chaves:
Primária: id_genero
Alternativa: nome

- Jogo (id_jogo, id_desenvolvedora, titulo, preco)

→ Propósito: cada elemento da tabela representa um jogo disponível na plataforma. Esse registro será usado tanto para a loja quanto para a biblioteca de uma conta. O nome do jogo é único.
→ Atributos: identificador único e dados satélite.
→ Chaves:
Primária: id_jogo
Alternativa: titulo
Estrangeiras: id_desenvolvedora, associa um jogo à sua desenvolvedora

- Jogo_Genero (id_jogo, id_genero)

→ Propósito: relaciona as tabelas jogo e gênero, permitindo que um jogo tenha mais de um gênero.
→ Atributos: somente os identificadores das instâncias que relaciona.
→ Chaves:
Primária: (id_jogo, id_genero)
Estrangeiras: id_jogo referencia jogo, id_genero referencia genero
→ Outras restrições:
on delete cascade garantem que quando um jogo ou gênero forem removidos da loja, a associação entre eles também é removida.

- Pedido (id_pedido, id_conta, data_pedido)

→ Propósito: representar um pedido feito por uma conta na loja.
→ Atributos: identificador único, a referência para a conta que realizou o pedido e a data da compra.
→ Chaves:
Primária: id_pedido
Estrangeira: id_conta referencia conta
→ Outras restrições:
on delete cascade garante que se uma conta for removida, pedidos atrelados a ela também são.

- Item_Pedido (id_pedido, id_jogo, preco_pago)

→ Propósito: permitir que um pedido tenha mais de um item. Conceitualmente, funciona como jogo_genero.
→ Atributos: os identificadores únicos dos elementos que relaciona e o preço pago pelo item.
→ Chaves:
Primária: (id_pedido, id_jogo)
Estrangeiras: id_pedido referencia pedido, id_jogo referencia jogo
→ Outras restrições:
on delete cascade garante que ao apagar um pedido, o registro dos itens de um pedido são deletados também. Se os itens de um pedido forem deletados, o pedido permanece no BD, permitindo resgatar histórico de compra mesmo que o item não esteja mais na loja.
check (preco_pago >= 0) garante que nenhum item será cadastrado com preço negativo.

❿ Biblioteca (id_conta, id_jogo, horas_jogadas, ultima_sessao)

→ Propósito: modelar a biblioteca de jogos de uma conta. Adiciona dados relativos ao jogo exclusivos a uma conta.

→ Atributos: os identificadores dos elementos que relaciona e dados satélite.

→ Chaves:

Primária: (id_conta, id_jogo)

Estrangeiras: id_conta referencia conta, id_jogo referencia jogo

→ Outras restrições:

```
ultima_sessao date default null
        check (ultima_sessao is null or ultima_sessao <= current_date),
```

permite a representação de um jogo adquirido mas nunca jogado, e evita que seja cadastrada última sessão além do dia atual.

`foreign key (id_conta, id_jogo) references biblioteca(id_conta, id_jogo) on delete cascade` também a referência é feita a Biblioteca, garantindo que um usuário precisa ter adquirido o jogo para poder avaliá-lo.

`on delete cascade` garantem que se uma conta for removida, sua biblioteca também é, além de garantir que uma conta não possa ter jogos que não existem mais.

❿ Avaliacao (id_jogo, id_conta, nota, comentario, data_avaliacao)

→ Propósito: representar uma avaliação feita por um jogador, deixada na página da loja de um jogo.

→ Atributos: identificadores dos elementos que relaciona e a avaliação, propriamente dita.

→ Chaves:

Primária: (id_jogo, id_conta)

Estrangeiras: id_jogo referencia jogo, id_conta referencia conta.

→ Outras restrições:

`check (nota in (1, 2, 3, 4, 5))` no nosso caso, as avaliações são como um sistema de estrelas, variando de 1 a 5 (inteiros).

`on delete cascade` garantem que se tanto o jogo avaliado ou a conta avaliadora forem deletados, a avaliação também é.